COMPANION TO THE COMPLETE REFERENCE

Operator's Field Guide

The how's & why's — written for you, not the lawyer

The Complete Reference tells you *what* the system is. This guide tells you *why each piece exists*, *how to actually run it*, and *where it tends to break*. This is back-stage — your thinking document.

Read it once end-to-end · then dip into a single entry when a step feels unclear

How to read this guide

Every entry below uses the same three lenses, so you can scan for the one you need:

WHY — the reasoning behind the design choice (this is the part the clean reference leaves out). **HOW** — what you concretely do when you're in it. **WATCH** — the failure mode that quietly kills the step if you let it.

A The Two Splits Everything Hangs On

READ THIS FIRST

Before the seven steps make sense, two distinctions have to click. Almost every "where does this go?" question is really one of these two.

CORE PRINCIPLE

Separate the space where you *think and learn* from the space where you *perform and record* — and separate *conversation you'll consume once* from *structure you'll reuse*. Keep those four quadrants clean and the whole system runs itself.

1 Personal vs. Teams — AUDIENCE & MODE

WHY Two different reasons to split. **(1) Record:** Teams is the shared, case-bound, official trail — anything there is part of the engagement. Personal is yours alone. **(2) Mode:** Personal is where you think freely, try bad ideas, and feel things. Teams is where you perform and document. If you mix them, you do one of two damaging things — dump your mess into the official record, or self-censor your real thinking because it's in a shared space. The split lets you be fully messy *and* fully professional, just in different rooms.

HOW Ask: *"Would I be comfortable with the lawyer or the team scrolling this?"* If the honest answer is "not yet / not all of it" → Personal. If it's case fact meant for the shared trail → Teams.

WATCH The sneaky failure is doing your *learning* inside Teams because it's "more efficient." It isn't — it pollutes the case record and makes you guarded. Build and debrief stay Personal, always.

2 Cowork vs. Chat REUSE TEST

WHY **Chat** is linear — a conversation you have, consume, and move past. **Cowork** is structural — executable artifacts, code, templates, things you'll pull *into* other work later. The mechanical difference is durability: a Chat is a moment; a Cowork object is an asset. Put a reusable asset in Chat and it gets buried in scrollback. Put a one-off conversation in Cowork and it clutters your library with noise.

HOW One question decides it: *"Will I want to reference this object from inside another project later?"* Yes → Cowork. No, it's thinking/feedback → Chat.

WATCH Don't let valuable structure (a playbook, a template, a working script) live out its life in a Chat thread. The instant something becomes reusable, it wants to be promoted into Cowork — that's literally what Step 2 is.

Messy code, half-built, exploring	→	**Personal · Cowork**
Proven asset you'll reuse	→	**Library (Workflows & Architecture)**
Judgement / "does this look right?"	→	**A Chat project**
Processing how it went, for you	→	**Personal · Chat**
Factual record of a real run	→	**Teams (Chat for notes, Cowork for the run)**

B The Seven Steps — Why Each One Earns Its Place

THE ENGINE

The order isn't arbitrary. Each step exists to protect the one after it. Read each entry's **WHY** as "what goes wrong if this step didn't exist."

THE SHAPE OF THE LOOP

Build → prove → gate → run → record facts → process feeling → fold back. It's a *learning loop*, not a checklist. The entire payoff is that disclosure N+1 costs less than disclosure N — because every cycle leaves the Vision sharper and the Library richer.

0 Vision — the North Star

ABOVE EVERYTHING

WHY

Every tactical call below — *"build this?", "automate that?", "is this good enough to ship?"* — needs a reference point. Without a North Star you optimize **locally**: each disclosure gets marginally better while you quietly drift from what actually matters. The Vision is the tiebreaker. It sits above the seven steps because it's the thing you check when two options both look reasonable.

HOW

Write it down in one short paragraph and pin it in the Library. Phrase it as outcomes, not tasks. Re-read it at Step 7 and let it evolve — a North Star that never changes probably isn't being used.

WATCH

A vision you wrote once and never look at is decoration. If you're not consulting it when you make hard calls, it isn't doing its job.

1 Build & Debug

PERSONAL · COWORK

WHY

Building is messy by nature — dead ends, half-thoughts, code you'd never show anyone. That needs **psychological safety**: a room where bad work is expected and fine. Do it in the Teams space and two things break — you pollute the shared record, and you self-censor, which kills the experimentation that makes building work. It's Cowork because the *output* is an executable artifact, not a chat.

HOW

Treat it as a sandbox. Iterate hard, break things, don't clean up yet, don't promote anything. The only goal of Step 1 is: **a thing that works once** — even if it's ugly.

WATCH

Polishing too early. Don't make it pretty here — pretty is Step 2's job. And resist using it on a real case before it's proven; the sandbox is not the run.

2 Promote to Library

→ WORKFLOWS & ARCHITECTURE

WHY

"It worked once in my sandbox" is *not* "it's a reusable asset." Promotion is the deliberate act that converts the two. Without an explicit promotion gate your Library never accumulates — you rebuild the same thing every disclosure and never compound. The gate also forces an honest question: *"Is this actually proven, or did it just happen to work once?"*

HOW

Take the thing that worked, **strip the case-specific bits**, generalize it, and write a short "when & how to use this" note next to it. Then place it in Workflows & Architecture. That note is what makes it findable and trustable later.

WATCH

Promoting too eagerly. If it's only run once, it's a candidate, not a proven asset — say so in the note. A library full of unproven "assets" is as useless as no library.

3 Pre-run Review TEAMS · CHAT

WHY Execution (Step 4) is the costly, consequential part — a real run on a real case. Catching a problem *before* the run is an order of magnitude cheaper than catching it mid-run. This is your **quality gate**. It's Chat because reviewing is judgement and conversation — *"does this look right? what could go wrong?"* — not code production. It's Teams because it's case work and belongs in the shared trail.

HOW Walk the plan end-to-end against the playbook. Sanity-check the inputs. Ask the one productive question: *"What would make this fail?"* — and resolve those before you touch Step 4.

WATCH Skipping the gate when you feel confident. Confidence is exactly when sloppy runs happen. The review is cheap; pay it every time.

4 Execution TEAMS · COWORK

WHY This is the actual run — executable work on real artifacts, so it's Cowork; it's the case, so it's Teams. It's kept separate from Build for a reason: **you execute using proven Library assets, you don't improvise here.** Execution is performance, not experimentation.

HOW Run the proven playbook against the real case. Follow it; don't redesign it mid-run.

WATCH Catching yourself *building* during execution. That's a signal something wasn't actually ready — don't fix the system live. Note the gap and carry it to the debrief, then back into Build/Library.

5 Audit TEAMS · CHAT

WHY Right after the run the details are freshest — and they decay fast. The audit captures **what mechanically happened**: what ran, what broke, what took longer than planned, what the outputs were. It's deliberately kept separate from the debrief because *facts and feelings are different modes* and mixing them muddies both. The audit is the objective record; it's Teams Chat because it's case fact, conversational.

HOW Log the concrete sequence: steps as they actually went, every deviation from plan, every anomaly. No interpretation yet — just the nuts and bolts.

WATCH Letting emotion leak in ("that was a nightmare"). Park feelings for Step 6. And don't postpone — an audit written tomorrow has already lost half its detail.

6 Debrief — the trip home PERSONAL · CHAT

WHY This one is about **you**: what you learned, what clicked, what frustrated you, what you'd change. That's personal processing, not team record — so it's Personal. It's the "trip home after the audit": you file the facts at the office (Teams), then go home (Personal) and process how it actually went for you. This is where the system's *learning* lives. Skip it and you execute forever without ever getting better.

HOW Answer four questions honestly: *What surprised me? What did I learn? What do I want to change? How did it feel?* Feelings are data here — frustration usually points at a real gap in the playbook.

WATCH Treating the debrief as optional because "the work is done." The run is done; the *compounding* hasn't happened yet. No debrief, no improvement.

7 Fold Lessons Back → VISION + LIBRARY

WHY Lessons that aren't written into something **durable** evaporate within days. This is the step that makes the whole system compound. It feeds **Step 0 (Vision)** and **Step 2 (Library)** specifically because those are the only two *persistent* stores — everything else is transient, per-disclosure. Folding back is how a one-time insight becomes a permanent upgrade.

HOW Make concrete edits: update the pinned Vision if your direction shifted; update or add a playbook in the Library; promote any new proven asset. The test is whether the *next* disclosure will actually start from a better place.

WATCH Debriefing and then doing nothing with it. A debrief that doesn't change the Vision or the Library is just journaling — the loop stays open.

Why the loop has this exact shape

Each step protects the next. Build gives Promote something proven to lift. Promote gives Execution a real asset instead of improvisation. The Pre-run gate protects the costly run. Audit captures facts before they fade so the Debrief has something true to reflect on. The Debrief produces the learning that Fold-Back makes permanent — which raises the floor for the next Build.

```
Build → Prove → Gate → Run → Record → Reflect → Fold back → (better) Build
```

C Why the Action Plan Runs In That Order

SEQUENCING LOGIC

The phases aren't just a to-do list — the order is the whole point. Here's the reasoning behind each handoff.

WHY CLEANUP COMES FIRST (PHASE 1)

You can't build a clean system on a messy foundation — start the new workflow on top of disorganized files and you simply **import the chaos**. Cleanup makes room for the new structure to live. And the file audit itself is informative: it shows you what you actually have and use, which tells you what the Library should hold.

WHY DOCUMENT THE WORKFLOW BEFORE RUNNING IT (PHASE 1 → 2)

Disclosure Two is partly a **test of the system itself**. If the workflow isn't written down, you can't tell whether a problem came from the case or from the system — the two get tangled. Documentation makes the system *inspectable*, so the first real run can debug the process, not just the case.

WHY THE LAWYER-FACING PIECES ARE BUILT IN PHASE 2, BEFORE THE RUN

The Playbook Template and the "Lawyer Ideas & Requests" project are the **front-stage**. Setting them up before Disclosure Two means that when you run, you already know what the clean deliverable looks like — so you can capture it *as you go* instead of reconstructing it afterward from memory.

WHY UPDATE THE VALUE TIMELINE WITH REAL DATA AFTER DISCLOSURE TWO (PHASE 3)

Your first timeline is a guess. Guesses don't persuade — yours or the lawyer's. The moment you have **one real number** (how long Disclosure Two actually took under the system), the whole case for the system stops being a pitch and becomes evidence. Replace estimates with reality the instant you have it.

WHY AUTOMATION IS LAST, AND GATED ON 2–3 MANUAL RUNS (PHASE 4)

This is the big one — see Section E. In short: you can only automate a process that has **stopped changing**. Two to three manual runs is roughly when the playbook stabilizes. Automate before that and you're building on a moving target, so the build keeps breaking as the process shifts under it.

D The Concepts, With the Reasoning Restored

DEEP CUTS

§ The Lawyer Playbook Separation

WHY

Two audiences need two different artifacts. The lawyer needs *confidence and clarity* — a clean deliverable, a simple playbook, a status. They do not need (and shouldn't see) your iterations, debates, and dead ends; that mess *erodes* their confidence and burns their attention. Meanwhile **you** need that exact mess — the full record of iterations is where your learning lives. So you deliberately maintain both: a clean **front-stage** for them, a rich **back-stage** for you.

WATCH

Collapsing the two — in either direction. Dumping back-stage mess on the lawyer destroys confidence. Throwing away your back-stage to "look clean" destroys your learning. Keep them separate on purpose.

FRONT-STAGE · THE LAWYER SEES

Clean deliverable · simple one-page playbook · status updates. **Built for confidence.**

BACK-STAGE · ONLY YOU SEE

Every iteration · the debates · the cleanup · the learning. **Built for compounding.**

§ The Value Timeline

WHY

It's your **justification engine**, pulling double duty. For *you* it's motivation — proof the front-loaded effort pays off, so you don't quit during the expensive first run. For the *lawyer* it's ROI — a legible picture of what the system saves. Three timelines side by side make the investment visible in a way a paragraph never could.

HOW

Hold the three lines in mind as you work:

1 · NO SYSTEM

12–15 hrs per disclosure — and it stays there forever. The counterfactual.

2 · WITH SYSTEM

D1 **12h** (build) → D2 **6h** (refine) → D3 **4h** (execute). Front-loaded, then it pays.

3 · WITH AUTOMATION

D4+ **~30 min** to trigger; the system runs itself. The payoff curve.

WATCH

Leaving it as estimates. The chart only persuades once it's real — overwrite the guesses with measured hours as soon as Disclosure Two gives you a true number.

§ The Library (Workflows & Architecture) as the system's memory

WHY

Of everything in the system, only two things **persist** across disclosures: the Vision and the Library. Everything else is per-case and disposable. That's why both Promote (Step 2) and Fold-Back (Step 7) point *here* — the Library is the system's long-term memory. A thin library means every disclosure starts near zero; a rich one means each starts further along. It's also where agents will eventually live, because an agent is just an executable Library asset.

HOW

Keep it curated, not a junk drawer. Pin the workflow diagram and the conversation summary. Every asset gets a short "when & how to use" note. Prune things that proved wrong.

E The Automation Path — Why You Wait

THE LONG GAME

This is the most counterintuitive part of the system, so it gets its own deep treatment. The instinct is to automate early. The instinct is wrong, and here's exactly why.

THE ONE RULE

The agent doesn't *invent* the process — it *executes* the one you've already proven works. An agent is the automation version of a playbook that already exists. No stable playbook, no agent.

PHASE 1 · MANUAL

You run disclosures by hand. The real job here isn't just doing them — it's **learning, documenting, and debriefing** so the process becomes legible.

PHASE 2 · PLAYBOOK

After a few runs you have a **solid, stable** playbook — one that stopped changing between disclosures. That stability is the signal you're ready.

PHASE 3 · AGENT

Using the Agent SDK and your API key, you teach an agent to **follow that proven playbook**. It lives in the Library and runs what you already trust.

WHY AUTOMATING EARLY IS THE CLASSIC TRAP

If you automate a process you haven't proven, you **automate your mistakes** — and now they run faster, at scale, and you trust them less, not more. Worse, an unproven process is still *changing*: every disclosure you discover a better way. Automation built on a moving target keeps breaking, so all that build effort gets spent re-fixing the agent instead of doing disclosures. You'd be paying the highest-effort cost (building an agent) on the lowest-certainty input (a process you don't yet trust).

WHY 2–3 MANUAL RUNS IS THE THRESHOLD

One run proves nothing — it might've been luck or a tame case. By the second and third, you can see which parts are **stable** (same every time → safe to automate) and which are still **judgement calls** (different every time → keep human). That distinction is exactly the design spec for the agent: automate the stable spine, leave the judgement to you, and keep a review-and-approve checkpoint so you stay in the loop.

The direction is one-way

You move from manual proof, to a written playbook, to an agent — and never the reverse. Every attempt to jump straight to the agent skips the only thing that makes the agent trustworthy: a process you've already watched succeed with your own hands.

```
Manual proof ──► Playbook ──► Agent   (never the other way around)
```

Back-stage document — keep it in Personal / the Library. Revisit and edit it at every Step 7, the same as the Vision.